

Week 1: Software Engineering Fundamentals

WEEK 1 — ENGINEERING MINDSET BOOTCAMP

DAY 1 — SYSTEM REVERSE ENGINEERING + NODE & TERMINAL

1 Create a script `sysinfo.js` that prints:

- Hostname
- Available Disk Space (GB)
- Open Ports (top 5)
- Default Gateway (network)
- Logged-in users count

2 Create 3 shell aliases

Store in `.bashrc/.zshrc`:

```
alias gs="git status"
alias files="ls -lha"
alias ports="lsof -i -P -n | grep LISTEN"
```

3 Run Node program + log runtime metrics:

```
process.cpuUsage()
process.resourceUsage()
```

Store in `/logs/day1-sysmetrics.json`

Deliverables

- `sysinfo.js`
- `.bashrc` or `.zshrc` snippet screenshot
- `logs/day1-sysmetrics.json`

DAY 2 — NODE CLI & CONCURRENCY

New Exercises

Build a CLI tool:

```
stats.js --lines <file> --chars <file> --words <file>
```

✓ Count total:

- Characters
- Lines
- Words
 - ✓ Process **3 files in parallel**
 - ✓ Output performance report:

```
{  
  "file": "data1.txt",  
  "executionTimeMs": 51,  
  "memoryMB": 14.3  
}
```

Bonus:

Add flag to **remove duplicate lines** and write to:

```
output/unique-<filename>
```

Deliverables

- `stats.js` CLI
 - `/logs/performance*.json`
 - Output files with uniqueness processing
-

DAY 3 — GIT MASTERY (RESET + REVERT + CHERRY-PICK + STASH)

New Exercises

① Setup repo with 10 commits:

- Introduce **syntax error** in commit 5
- Use **git bisect** to detect breaking commit → record

② Create a **release** branch from older commit:

```
release/v0.1
```

③ Use **cherry-pick** to bring only **important changes** from main → release

④ Use **stash** while switching branches + restore cleanly

⑤ Document:

- Bisect command logs
- Cherry-pick result
- Stash scenario

Deliverables

- `bisect-log.txt`
- `cherry-pick-report.md`
- `stash-proof.txt`
- Commit graph screenshot

DAY 4 — HTTP / API FORENSICS (cURL + POSTMAN)

New Exercises

① Use cURL to fetch **GitHub API**

```
curl -v https://api.github.com/users/octocat
```

Extract & log:

- Rate-limit remaining
- ETag
- Server header

② Pagination fetch:

```
https://api.github.com/users/octocat/repos?page=1&per_page=5
```

Document:

- Link headers
- Navigating pages

③ Create POSTMAN collection testing:

- GET GitHub user
- GET repos × 3 pages

④ Build HTTP Server with:

- `/ping` return timestamp
- `/headers` return request headers
- `/count` maintain counter in memory

Deliverables

- `curl-headers.txt`
- `pagination-analysis.md`
- POSTMAN exported collection `.json`
- `server.js`

DAY 5 — AUTOMATION & MINI-CI PIPELINE

New Exercises

① Build script: `healthcheck.sh`

- Pings your server every 10s
- Logs failures to `/logs/health.log`

② Pre-commit validation:

- ✓ Ensure `.env` file **does not exist** in Git
- ✓ Ensure **JS is formatted**
- ✓ Ensure log files are ignored

③ Packaging:

```
bundle-<timestamp>.zip
```

Include:

- `src/`
- `logs/`
- `docs/`
- `checksums.sha1`

4 Cron/Task scheduling:

Every 5 min → run `healthcheck.sh`

Deliverables

- `healthcheck.sh`
- Husky pre-commit hook screenshot (failed & passed)
- `bundle*.zip`
- `checksums.sha1`
- Screenshot of scheduled cron job

Week 2: Frontend (HTML, CSS, JS)

Week 2 — Frontend Fundamentals (HTML + CSS + JavaScript)

Objective:

Enable you to build responsive UI layouts and manipulate DOM with JavaScript using ES6 concepts.

Week-2 Topics

HTML

- Semantic HTML5 structure
- `<header>` `<footer>` `<nav>` `<main>` `<section>` `<article>`
- Forms, validation, accessibility basics
- `<canvas>` & media (`video`, `audio`)

CSS / Styling

- CSS Selectors (attribute, sibling, nth-child)
- Box model, specificity, units (`rem`, `vw`, `vh`)
- Flexbox, Grid (2D layouts)
- Responsive design (mobile-first, media queries)
- Animations, transitions

JavaScript (ES6+)

- Variables `let` vs `const`
- Arrays/objects (map, filter, reduce)
- Arrow functions / destructuring / spread operator
- DOM manipulation
- Event listeners
- Modular JS: splitting code into functions/modules

DAY 1 – HTML5 + Semantic Layout

♦ Learning Outcomes:

- Understand page structure
- Master semantic tags and responsive layout scaffolding

Topic	Activity
HTML Fundamentals	tags / structure / metadata
Semantic HTML5	Build semantic layout (no <code><div></code> allowed)
Forms & media	Build form (input, select, validation) and embed media
Accessibility	ARIA labels, alt text, tab index
Documentation	Write README with learnings

 **Exercise:** Build a *Blog Page* using only semantic HTML, no CSS. **Image for reference -** <https://www.figma.com/proto/XVgDL7iGFQaWNCQjqW3hhN/Free-Blog-Website-Template--Community-?node-id=401-147&t=HhIAtSENyqirHvHH-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

 Deliverable: `blog.html`

DAY 2 – CSS Layout Mastery (Flexbox + Grid)

♦ Learning Outcomes:

- Modern responsive layout using Flexbox & CSS Grid

Topic	Activity
CSS Selectors & Specificity	selector challenges
Flexbox	Build navbar + hero section
CSS Grid	Product grid layout (2/3/4 col based on width)
Responsive approach	Convert desktop → mobile-first

 **Exercise:** Replicate a **UI screenshot given by mentor** using Flex/Grid. **Image for Reference -**

https://cdn.prod.website-files.com/672e5a09a305c0f18b07ed8a/6753168922cb7762afcf50a2_3.png

✓ Deliverable: `index.html` + `style.css` + screenshots of comparison

DAY 3 – JavaScript ES6 + DOM Manipulation

♦ Learning Outcomes:

- Modern JS (ES6) + manipulating DOM without frameworks

Topic	Activity
Variables/functions	<code>let/const</code> , arrow functions
Arrays/objects	<code>map</code> , <code>filter</code> , <code>reduce</code> mini-challenges
DOM manipulation	build navbar toggle, dropdown, modal
Event listeners	build counter + key events

✏ **Exercise:** Build an **interactive FAQ accordion** using JS (click to expand).

https://codingstella.com/wp-content/uploads/2024/02/img_6092b008464f3.jpg

✓ Deliverable: `/js-dom-practice/*`

DAY 4 – JS Utilities + LocalStorage Mini-Project

♦ Learning Outcomes:

- Modular JS functions
- LocalStorage persistence

Topic	Activity
Debugging DevTools	breakpoints, watch

Custom JS utilities	<code>debounce</code> , <code>throttle</code> , <code>groupBy</code>
LocalStorage project	Build Todo app (persist on refresh)
Error handling	try/catch + error boundary folder (<code>logs/errors.md</code>)



Exercise:

Build **Todo App** with **LocalStorage** persistence

(Add → Edit → Delete → Persist after refresh)



Deliverable: `todo-app/`

DAY 5 – Capstone UI + JS Project

♦ Learning Outcomes:

Combine everything from HTML + CSS + JS into a working UI

Project: Build a mini “E-commerce product listing page”

Requirements:

- Use fetch API:
`https://dummyjson.com/products`
- Display product cards (title, image, price)
- Search bar (filter products)
- Sort (high → low price)
- Mobile responsive layout

Image for Reference -

<https://codehim.com/wp-content/uploads/2021/09/bootstrap-5-ecommerce-product-list-navbar-and-hover-effects.png>

Activity	Output
Project setup	folder + planning
UI using HTML + CSS	skeleton ready
JS fetch + rendering + search	functional UI

Final touches/responsive/polish	proper layout
------------------------------------	---------------

✓ Deliverables:

- Repo: `week2-frontend`
- Pages inside repo:
 - `/index.html`
 - `/products.html`
- README containing **screenshots + what you learned**

Week 3: Frontend Advanced

Week 3: Advance Frontend (Next.js + TailwindCSS)

Objective:

Train interns to build modern, production-grade frontends using:

- Next.js fundamentals (App Router / Pages / Routing / Layouts / Image optimization / SSR vs CSR basics)
 - TailwindCSS (utility-first design + responsive components + reusable design system)
-

Topics (Week-3)

Next.js Fundamentals

- File-based routing
- `app/` directory structure (Next 15+)
- Layouts & nested layouts
- Page navigation (`Link`, `useRouter`)
- Static vs client components ("`use client`")
- Image optimization (`next/image`)
- Fonts optimization (`next/font`)
- Metadata & SEO tags (`<Head />` or `metadata` config)

TailwindCSS Fundamentals + Design System

- Utility-first styling
- Responsive design (`sm`, `md`, `lg`, `xl`)
- Flexbox + Grid using Tailwind
- Spacing system (`p-x`, `m-x`, `gap-x`)
- Typography system
- Custom theme config in `tailwind.config.js`
- Component composition mindset (atoms → molecules → sections → pages)

Component Architecture + UI Thinking (in Next.js + Tailwind)

- Reusable component system

Folder structure:

```
/components  
/app  
/styles
```

✓ Week-3 Exercises (Hands-on Deliverables)

Area	Exercise
Tailwind Setup	Convert Figma/screenshot into Tailwind components
Component System Design	Build Button, Card, Modal, Avatar, Badge
Next.js Routing	Multi-page project with shared layout
UI Challenge	Replicate a SaaS landing page
Reusable Component Library	Create <code>ui/</code> library folder with export index
SEO	Metadata config + Image optimization

DAY 1 — TailwindCSS + UI System Basics

◆ Topics:

- Install Tailwind in Next.js
- Utility classes, spacing, colors, fonts
- Custom theme configuration

◆ Exercise:

Build a **Dashboard Layout skeleton** (header + sidebar)

Image for reference (Create header and Sidebar only) -

<https://www.figma.com/proto/JdoKvxwRE44a4h3bnpu3ZX/Purity-UI-Dashboard---Cakra-UI-Dashboard--Community-?node-id=29-2&t=bm1QziJKazGeZkad-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

◆ Output:

- `/app/layout.jsx`
 - `/components/ui/Navbar.jsx`
 - `/components/ui/Sidebar.jsx`
-

DAY 2 — Tailwind Advanced + Component Library

◆ Topics:

- Flex + Grid with Tailwind
- Components mindset (atomic design)
- Reusable components with props

◆ Exercise:

Build a component library in:

`/components/ui/`

- `Button.jsx`
- `Input.jsx`
- `Card.jsx`
- `Badge.jsx`

- Modal.jsx

Image for reference (Build the full layout using reusable components, reusing the Day-1 UI component sidebar and header) -

<https://www.figma.com/proto/JdoKvwxwRE44a4h3bnpu3ZX/Purity-UI-Dashboard---Chakra-UI-Dashboard--Community-?node-id=29-2&t=bm1QziJKazGeZkad-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

♦ **Output:**

- `/components/ui/*`
 - `UI-COMPONENT-DOCS.md` (Usage examples)
-

DAY 3 — Next.js Routing + Layout System

♦ **Topics:**

- Next.js routing (`app/page.js`, `/about/page.js`, `/dashboard/page.js`)
- Nested layouts
- Shared navigation across pages
- Difference: **Client Component** vs **Server Component**

♦ **Exercise:**

Build a multi-page structure:

`/` (landing page)

`/about`

`/dashboard`

`/dashboard/profile`

Image for reference - (Use the day 2 complete UI and create static page to achieve routing and layout system)

<https://www.figma.com/proto/JdoKvwxwRE44a4h3bnpu3ZX/Purity-UI-Dashboard---Chakra-UI-Dashboard--Community-?node-id=29-2&t=bm1QziJKazGeZkad-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

♦ **Output:**

- Folder structure working with nested layouts
-

DAY 4 — Dynamic UI + Image Optimization

♦ Topics:

- next/image (optimized image rendering)
- Responsive images + optimization
- Typography & SEO improvements
- Animations using Tailwind + Framer Motion (optional)

♦ Exercise:

Build a **responsive landing page** (like SaaS product page)

Sections:

- Hero section
- Features grid
- Testimonials (cards)
- Footer

Image for reference - (Use the code completed up to Day 3 and make it responsive with Tailwind classes. Also, implement on-page SEO tags.)

<https://www.figma.com/proto/JdoKvxwRE44a4h3bnpu3ZX/Purity-UI-Dashboard---Chakra-UI-Dashboard--Community-?node-id=29-2&t=bm1QziJKazGeZkad-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1>

♦ Output:

- `/app/page.jsx`
 - `screenshots/landing-page.png`
-

DAY 5 — Capstone Mini Project (No backend)

♦ Project:

Build a **Full Multi-Page UI** in Next.js + Tailwind, no backend.

Pages:

```
/login      (static form)
/dashboard  (cards and widgets UI)
/dashboard/users (table listing mocked data)
/dashboard/profile
```

Design references :

Login Page -

https://img.freepik.com/free-vector/simple-white-login-form_1017-7984.jpg

Dashboard - Use the day 4 exercise for dashboard page.

Users Listing -

<https://www.figma.com/proto/JdoKvxwRE44a4h3bnpu3ZX/Purity-UI-Dashboard--Chakra-UI-Dashboard--Community-?node-id=29-2&t=bm1QziJKazGeZkad-0&scrolling=auto&content-scaling=fixed&page-id=0%3A1>

Profile page -

<https://www.gravitykit.com/wp-content/uploads/2023/10/CleanShot-2025-06-26-at-12.53.16@2x-1024x503.png.webp>

Mandatory:

- Component reuse from `/components/ui`
- Clean routing structure
- Mobile responsive UI

♦ Output:

- Project repo: `week3-next-tailwind-frontend`
- README.md must include:
 - Screenshots
 - Folder structure
 - Components list
 - Lessons learned

Week 4: Backend (Node + Express + MongoDB)

WEEK 4 — BACKEND SYSTEMS & PRODUCTION ENGINEERING

(Node.js + Express + MongoDB + API Architecture + Security + Scalability)

WEEK 4 ADVANCED OBJECTIVES

Interns will demonstrate **independent backend engineering capability** by:

- Designing **custom backend system architecture**
- Implementing **strict separation of concerns** with enforced contracts
- Applying **database optimization & query strategies**
- Building **defensive, production-safe APIs**
- Introducing **observability & async workflows**
- Producing **auditable documentation & artifacts**

This week evaluates **thinking + execution**, not memorization.

DAY 1 — BACKEND SYSTEM BOOTSTRAPPING & LIFECYCLE

♦ Learning Outcomes:

- Node.js runtime lifecycle (not just event loop)
- Environment-driven bootstrapping
- Controlled startup & graceful shutdown
- Dependency orchestration

♦ Mandatory Folder Structure (No shortcuts)

```
src/  
  config/
```

```
loaders/  
models/  
routes/  
controllers/  
services/  
repositories/  
middlewares/  
utils/  
jobs/  
logs/
```

◆ Topics to Learn

- Event loop phases
- Node clustering
- Config loader + environment isolation
- Express advanced bootstrapping

◆ Exercise

Build:

1. **App loader** (loads Express, middlewares, DB, routes in order)
2. **Config loader** supporting:

```
.env.local  
.env.dev  
.env.prod
```

3. **Startup logs** using Winston/Pino:

```
✓ Server started on port X  
✓ Database connected  
✓ Middlewares loaded  
✓ Routes mounted: 23 endpoints
```

◆ Deliverables:

```
/src/loaders/app.js
/src/loaders/db.js
/src/utils/logger.js
ARCHITECTURE.md
```

DAY 2 - DATA DESIGN & QUERY PERFORMANCE (NON-CRUD)

◆ Learning Outcomes:

- Schema design for **read-heavy systems**
- Query cost analysis
- Data lifecycle rules

◆ Topics

- Embedded vs Referenced schema
- TTL indexes
- Sparse + compound indexes
- Pagination strategies (skip/limit vs cursor)

◆ Exercise

Build **Account** & **Order** schemas with:

- Pre-save hook (hash password or preprocess)
- Virtual fields (fullName or computed rating)
- Compound index: { **status: 1, createdAt: -1** }
- Field validation & transformations

Implement the repository pattern:

```
AccountRepository.create()
AccountRepository.findById()
AccountRepository.findPaginated()
AccountRepository.update()
AccountRepository.delete()
```


♦ **Deliverables:**

```
/models/Account.js  
/models/Order.js  
/repositories/account.repository.js  
/repositories/order.repository.js
```

- Index analysis screenshot from MongoDB Compass
-

DAY 3 — QUERY PIPELINES & FAILURE-SAFE APIs

♦ **Learning Outcomes:**

- API behavior under complex filters
- Dynamic filters, sorting, and Controlled soft-deletion
- Unified error contracts

♦ **Topics**

- Controller → Service → Repository flow
- Complex filters:

GET

```
/products?search=phone&minPrice=100&maxPrice=500&sort=price:desc&tags=apple,samsung
```

Soft deletes (flag + timestamp)

Advanced error handling:

- Typed errors
- Error codes
- Centralized error middleware

♦ **Exercise**

Build Product API with:

- Dynamic search engine (regex + OR/AND conditions)
- Filtering + sorting + pagination
- Soft delete with:

```
DELETE /products/:id → marks deletedAt  
GET /products?includeDeleted=true
```

- Global error formats:

```
{ success: false, message, code, timestamp, path }
```

♦ **Deliverables:**

```
/controllers/product.controller.js  
/services/product.service.js  
/middlewares/error.middleware.js  
QUERY-ENGINE-DOC.md
```

DAY 4 — API DEFENSE & INPUT CONTROL

♦ **Learning Outcomes:**

- Attack surface reduction & sanitize APIs
- Validation as first-class logic
- Rate limiting
- Input sanitization

Mandatory Protections

- Payload whitelisting
- Schema-level validation

- Request throttling
- Header hardening
- Query sanitization

♦ **Topics:**

- Preventing:
 - NoSQL Injection
 - XSS
 - Parameter pollution
- Joi / Zod validation
- Helmet + CORS
- Rate limiting with `express-rate-limit`

♦ **Exercise**

1. Build robust **validation schema** for User + Product.
2. Add global:
 - rate limiting
 - CORS policy
 - Helmet security headers
 - Payload size limits
3. Write **security test cases** (manual)

♦ **Deliverables:**

```
/middlewares/validate.js  
/middlewares/security.js (helmet, rate-limit, cors)  
SECURITY-REPORT.md (must show: vulnerabilities tested & results)
```

DAY 5 — ASYNC WORKERS, OBSERVABILITY & RELEASE READINESS

♦ **Learning Outcomes:**

- Async background jobs
- Structured logging
- Postman documentation
- Production-ready backend thinking

♦ **Topics:**

- Job queue design using:
 - Bull / BullMQ (Redis) (or in-memory fallback)
- Logging patterns (correlation IDs)
- Request tracing
- API documentation (Postman/Swagger)

♦ **Exercise**

Implement:

Background job

- Job: email notification or report generation
- Queue: BullMQ
- Retry + backoff
- Worker process & logs

Request tracing

- Every request gets **X-Request-ID**
- Logs grouped by request ID

API Documentation

- Auto-generate using Swagger OR produce a Postman Collection
- Include folder-level environment variables

Deploy-ready folder **prod/**

- **ecosystem.config.js** (PM2)
- **.env.example**

♦ **Deliverables:**

```
/jobs/email.job.js
/utils/tracing.js
/logs/*.log
Postman Collection Export
DEPLOYMENT-NOTES.md
```

WEEK-4 COMPLETION REQUIREMENTS

Skill Area	Requirement
Architecture	Layered structure + loaders
DB Modeling	Indexes + hooks + relations
API Engine	Pagination + sorting + filters
Security	All middlewares + vulnerability checks
Job Queues	Working background job
Documentation	Postman collection + diagrams

EXPECTED OUTCOME AFTER WEEK-4

Interns can now:

- ✓ Design backend systems independently
- ✓ Think beyond CRUD
- ✓ Anticipate production failures
- ✓ Secure APIs defensively
- ✓ Build observable services

Week 5: Server & Deployment

Week 5: Server Side Foundations with Docker & DevOps Basics

New Objective:

Introduce interns to how production servers actually run applications. Using Docker, interns will set up local environments that mirror real deployment setups, including services, configurations, and runtime behavior. This helps them understand how code moves from a developer's machine to a production server.

Week-5 Topics

DAY 1 — Docker Fundamentals + Linux Internals

- Images, containers, volumes, networks
- Basic container OS operations:
 - Users, permissions, logs, processes inside containers
- Dockerfile basics

Exercise

- Container running Node app
- Enter container (like SSH):

```
docker exec -it <container> /bin/sh
```

Explore:

- `ls`, `ps`, `top`, disk usage, logs
- Document structure

Output

- Dockerfile
- `linux-in-container.md`

DAY 2 — Docker Compose + Multi-Container Apps

- Docker networking
- Volumes for persistent storage
- Compose for multiple services.

Exercise

Deploy:

client (React) + server (Node) + MongoDB

using one command:

```
docker compose up -d
```

Ensure:

- ✓ Server connects to Mongo via container networking
- ✓ Logs exposed correctly
- ✓ Volumes mapped

Output

- [docker-compose.yml](#)
- [service-architecture.md](#)

DAY 3 — NGINX Reverse Proxy + Load Balancing

- NGINX inside Docker
- Routing to internal containers
- Multiple backend replicas (load balancing simulation)

Exercise :

- Run **two instances** of backend
- Use NGINX container as reverse proxy:

/api → backend-service:3000

Note : Enable round-robin load balancing

Output

- [nginx.conf](#)
 - [reverse-proxy-readme.md](#)
-

DAY 4 — SSL + Self-Signed + mkcert + HTTPS

- SSL/TLS fundamentals
- Setting certs inside container
- HTTPS termination at NGINX

Exercise

- Generate self-signed cert using mkcert
- Force redirect HTTP → HTTPS
- Confirm lock icon in browser using local domain

Output

- Certificates + [ssl-setup.md](#)
 - HTTPS working screenshot
-

DAY 5 — CI-Style Deployment Automation + Capstone

Tools:

- Docker volumes
- Compose profiles
- [.env](#) config
- Log rotation
- Health checks

Capstone

Deploy a **full-stack app** in Docker with reverse proxy + HTTPS

Checklist:

- ✓ `docker-compose.prod.yml`
- ✓ Secrets in `.env` (not committed)
- ✓ Healthcheck in compose
- ✓ Container restart policy
- ✓ Deployment script:

Output

- Fully working application stack
- `production-guide.md`
- Demo recording (optional)

Week 6: MACHINE LEARNING ENGINEERING

WEEK 6 — MACHINE LEARNING ENGINEERING

(Data Science + Feature Engineering + Model Building + Deployment + Monitoring)

WEEK 6 OBJECTIVES

Interns will learn:

Professional ML pipeline architecture

Clean, modular ML system design (Data → Features → Model → Evaluation → Deployment)

Advanced feature engineering + selection techniques

Model optimization + regularization + hyperparameter tuning

Training pipeline orchestration

Model evaluation + explainability

Model deployment + monitoring

This week produces engineers who can work on **production ML systems**.

DAY 1 — DATA PIPELINE + EDA + PROJECT ARCHITECTURE

♦ Learning Outcomes:

ML project architecture

Dataset versioning

Exploratory Data Analysis (EDA)

Professional folder structure

Data preprocessing pipelines

♦ Mandatory Folder Structure (No shortcuts)

```
src/  
  data/  
    raw/  
    processed/  
    external/  
  notebooks/  
  features/  
  pipelines/
```

```
models/  
training/  
evaluation/  
deployment/  
monitoring/  
utils/  
config/  
logs/
```

◆ Topics to Learn

- Train/validation/test splitting strategy
- Handling missing values (mean/median/interpolate)
- Outlier detection (Z-score / IQR)
- Data scaling (StandardScaler / MinMaxScaler)
- Class imbalance (SMOTE/weights)
- Dataset versioning (DVC / folder hashing)

◆ Exercise

Build a **data loader + EDA pipeline** that:

1. Loads dataset from `/data/raw`
2. Cleans data (missing, duplicates, outliers)
3. Creates `/data/processed/final.csv`
4. Generates EDA report:
 - Correlation matrix
 - Feature distributions
 - Target distribution

- Missing values heatmap

- ✓ Data loaded
- ✓ Cleaned dataset saved
- ✓ EDA report generated

◆ **Deliverables**

/pipelines/data_pipeline.py
/notebooks/EDA.ipynb
/data/processed/final.csv
DATA-REPORT.md

DAY 2 — FEATURE ENGINEERING + FEATURE SELECTION

◆ **Learning Outcomes:**

Create meaningful features
Encoding strategies
Advanced transformations
Feature selection techniques

◆ **Topics**

- Categorical encoding (OneHot, Target, Label)
- Numerical feature transformations (log, sqrt, power)
- Date/time feature extraction
- Text vectorization (TF-IDF / embeddings)
- Feature selection:
 - Correlation threshold
 - Mutual information
 - Recursive Feature Elimination (RFE)

♦ Exercise

Build a feature engineering pipeline that:

- Encodes categorical features
- Normalizes numerical features
- Generates:
 - 10+ new features
- Applies feature selection
- Produces `X_train`, `X_test`, `y_train`, `y_test`

- ✓ Feature pipeline saved
- ✓ Feature importance plotted

♦ Deliverables

```
/features/build_features.py  
/features/feature_selector.py  
/features/feature_list.json  
FEATURE-ENGINEERING-DOC.md
```

DAY 3 — MODEL BUILDING + ADVANCED TRAINING PIPELINE

♦ Learning Outcomes:

Multi-model training
Pipeline building
Cross-validation
Overfitting control

♦ Topics

- Models:

- Logistic Regression
- Random Forest
- XGBoost / LightGBM
- Neural Network
- Cross-validation (k-fold)
- Overfitting vs underfitting
- Regularization (L1/L2)
- Model comparison

◆ **Exercise**

Create a unified training pipeline that:

- Trains 4 models
- Performs 5-fold cross-validation
- Outputs:
 - Accuracy
 - Precision
 - Recall
 - F1 Score
 - ROC-AUC

Save the best model automatically.

- ✓ Best model selected
- ✓ Metrics saved
- ✓ Confusion matrix plotted

◆ **Deliverables**


```
/training/train.py  
/models/best_model.pkl  
/evaluation/metrics.json  
MODEL-COMPARISON.md
```

DAY 4 — HYPERPARAMETER TUNING + EXPLAINABILITY + ERROR ANALYSIS

♦ Learning Outcomes:

Optimize a model like an ML engineer
Interpret model decisions
Perform deep error analysis

♦ Topics

- GridSearch / RandomSearch / Bayesian tuning (Optuna)
- SHAP values
- Feature importance
- Error clustering
- Bias/variance analysis

♦ Exercise

Implement:

- Hyperparameter tuning with Optuna/GridSearch
- Generate:
 - SHAP summary plot
 - Feature importance chart
 - Error analysis heatmap

- ✓ Improvement over baseline
- ✓ Explainability added

♦ Deliverables

`/training/tuning.py`
`/evaluation/shap_analysis.py`
`/tuning/results.json`
`MODEL-INTERPRETATION.md`

DAY 5 — MODEL DEPLOYMENT + MONITORING + MLOPS CONCEPTS (CAPSTONE)

♦ Learning Outcomes:

Deploy real ML systems
Monitor drift
Production-ready pipelines

♦ Topics

- Serving model using FastAPI/Flask
- Creating `/predict` endpoint
- Saving & loading with joblib/pickle
- Input schema validation
- Monitoring:
 - Data drift
 - Accuracy decay
- Logging predictions

♦ Exercise (Capstone)

Deploy model as an API:

```
POST /predict
{
  "feature1": value,
  "feature2": value,
  ...
}
```

Add:

- ✓ Prediction logging
- ✓ Request ID tracking
- ✓ Input validation
- ✓ Versioned model loading
- ✓ Basic dashboard (Streamlit optional)

Prepare:

- Dockerfile
- requirements.txt
- .env.example
- README.md

◆ Deliverables

```
/deployment/api.py
/deployment/Dockerfile
/monitoring/drift_checker.py
/prediction_logs.csv
DEPLOYMENT-NOTES.md
```

WEEK- 6 COMPLETION REQUIREMENTS

Skill Area	Requirement
Data Engineering	Clean, versioned datasets

Feature Engineering	Automated + selected features
Model Building	Multiple models trained
Optimization	Hyperparameter tuning
Explainability	SHAP + feature importance
Deployment	Working API
Monitoring	Drift detection
Documentation	Full reports

EXPECTED OUTCOME AFTER WEEK- 6

Interns can now:

- ✓ Design end-to-end ML pipelines
 - ✓ Engineer advanced features
 - ✓ Train, tune and compare models
 - ✓ Explain model behaviour
 - ✓ Deploy ML systems as APIs
 - ✓ Monitor model performance
 - ✓ Work in real-world ML teams
-

Week 7: API + Architectures

Week 8: DevOps Introduction

Week 9: GenAI Fundamentals

Week 10: Machine Learning Basics

Week 11: Fine-tuning & Model Serving

Week 12: Final Capstone Project

